Python Course 2025/26/2

Lecture 8: Web APIs, REST, DTOs, and Dependency Injection

Course coordinator: Dr Mate Tejfel

Lecturer: Zoltan Kegli

Lecture Overview

Agenda

- How a web app differs from a one-shot CLI program
- HTTP requests, responses, and REST-style API design
- Why we separate domain models, DTOs, services, and routes
- FastAPI, Pydantic, OpenAPI docs, and modern dependency patterns
- Testing APIs with `pytest`, `TestClient`, and dependency overrides

Learning objective

Move from "I can write Python code" to "I can expose that logic as a clean, testable web API with explicit contracts."

By the end, you can

Explain the request-response model, choose sensible REST paths and status codes, separate transport objects from business logic, and test an API without launching a real browser.

Web HTTP FastAPI Pydantic Testing

Lecture 8 turns our to-do app into a service other programs can talk to.

From CLI to Web Services

The same Python ideas, but a very different runtime model

Mental Model Shift: CLI App vs Web App

CLI app

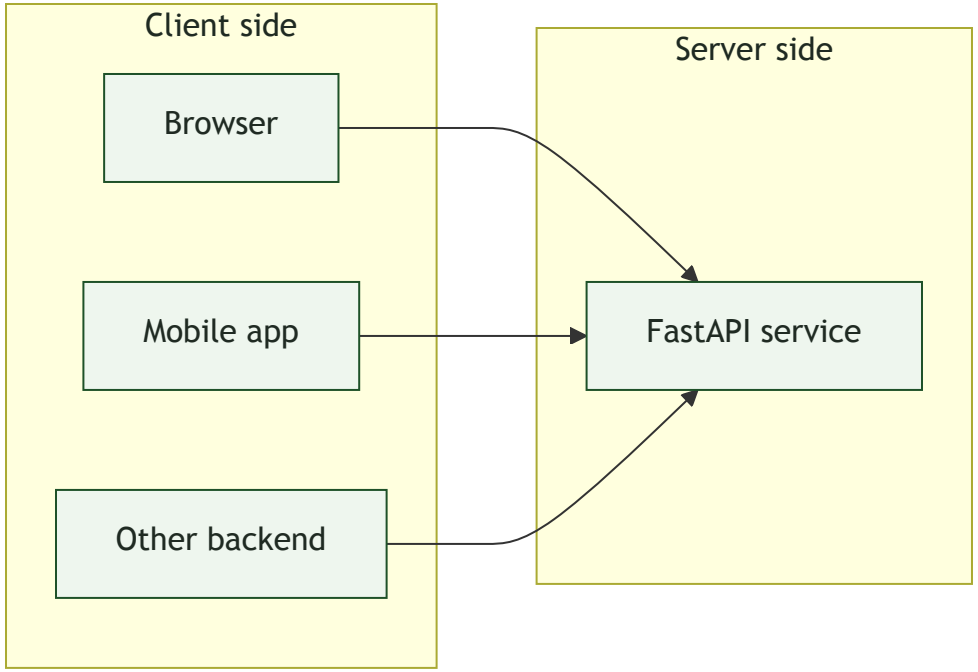
- Starts when `python ...` is executed
- Input often comes from arguments, files, or terminal prompts
- Output is usually text printed for one human user
- Stops as soon as the command finishes

Web service

- Starts once, then keeps listening for requests
- Input arrives as HTTP requests from many clients
- Output is usually structured data such as JSON
- Must survive many users, failures, and repeated calls

Same Python, different job: instead of solving one problem and exiting, the program becomes a long-running service.

The Client-Server Model

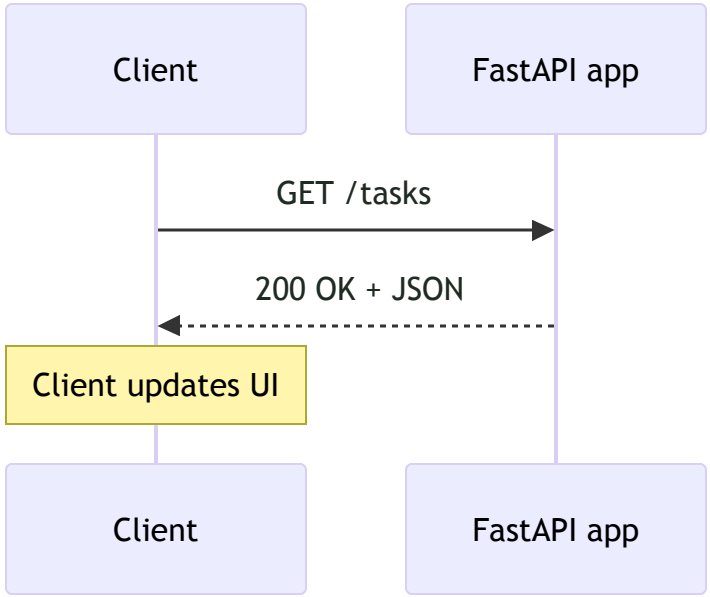


Who plays which role?

- **Client:** asks for data or sends commands
- **Server:** validates input, runs logic, returns a response
- **Network:** carries requests and responses between them

One API can serve a browser, a mobile app, and another service at the same time.

HTTP: One Request, One Response



Key HTTP idea

- The client sends a request message
- The server handles it and sends back one response
- HTTP itself is stateless: each request should stand on its own

Fun web fact

Opening one modern page can trigger dozens of HTTP requests for HTML, CSS, JavaScript, fonts, and images.

Checkpoint: From CLI to Web Services

Q1

Which statement best describes the main runtime difference between a CLI tool and a web API?

A) A web API usually keeps running and handles many requests over time.

B) A CLI tool must always return JSON.

C) A web API can only be used by browsers.

D) A CLI tool works only on Tuesdays.

Show Answer

Q2

If HTTP is stateless, what should a protected request generally include?

A) Enough information for the server to understand and authorize that request.

B) A memory dump of the previous request.

C) The entire source code of the client app.

D) A handwritten apology letter.

Show Answer

REST and HTTP Design

Turning nouns, verbs, and status codes into a usable contract

What Is an API?

Short definition

An API is an interface that lets one program ask another program to do something in a defined, predictable way.

Interface Contract Communication

Restaurant analogy

- **Client:** the customer placing an order
- **API:** the waiter carrying structured requests
- **Server logic:** the kitchen doing the real work
- **Response:** the finished dish delivered back

> The client does not need to know how the kitchen works internally. It only needs to know the menu and the rules.

Why APIs Matter

Separation

Frontends and backends can evolve independently as long as the contract stays stable.

Reuse

The same service can power a web UI, mobile app, tests, or another backend.

Interoperability

Programs written in different languages can still cooperate over shared protocols.

Real examples: payment gateways, map services, login providers, and AI APIs all let you reuse complex systems instead of rebuilding them.

REST: Resources, Methods, and a Contract

Think in resources

- `/tasks` means the whole collection
- `/tasks/42` means one specific task
- The URL names the thing; the HTTP method names the action

Fun API fact

The term `REST` was coined by Roy Fielding in his 2000 PhD dissertation.

Path	Method	Meaning
`/tasks`	`GET`	List tasks
`/tasks`	`POST`	Create a new task
`/tasks/42`	`GET`	Read one task
`/tasks/42`	`PUT`	Replace one task (full update)
`/tasks/42`	`PATCH`	Partially update one task
`/tasks/42`	`DELETE`	Remove one task

Real systems are often pragmatically RESTful: consistency matters more than purity debates.

Good Paths Feel Like Nouns

Prefer	Avoid	Why
`GET /tasks`	`GET /getTasks`	The method already says "get".
`POST /tasks`	`POST /createTask`	Use the collection path for creating a new resource.
`PATCH /tasks/42`	`POST /updateTask/42`	Let the method express update semantics.
`DELETE /tasks/42`	`GET /deleteTask/42`	Do not hide destructive actions behind read-looking URLs.

> **Rule of thumb:** the path identifies the resource, the method tells the server what kind of action the client wants.

Anatomy of an HTTP Request

```
POST /tasks HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer <token>

{
  "description": "Buy milk"
}
```

Requests have a start line, headers, a blank line, and an optional body.

Read it top to bottom

- **Method:** what action is requested
- **Path:** which resource is targeted
- **Headers:** metadata such as format or auth
- **Body:** extra data, often JSON

Common pattern: `GET` often has no body, while `POST` and `PATCH` usually do.

HTTP Responses and Status Codes

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id": 1, "description": "Buy milk"}
```

Status line + headers + optional body; `204` means success with no body.

Same API, other outcomes

`GET /tasks/999` — nothing there

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{"detail":"Task not found"}
```

`POST /tasks` with bad JSON shape

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{"detail":[{"loc":["body","description"],"msg":"field required"}]}
```

`DELETE /tasks/1` — gone

```
HTTP/1.1 204 No Content
```

First digit = family: **2** success · **4** fix auth/request · **5** server / temporary outage.

Code	Fam.	Typical use
`200`	2xx	Read or update OK
`201`	2xx	Created (`Location` often set)
`204`	2xx	Success, no response body
`400`	4xx	Bad / unusable request
`401`	4xx	Not authenticated
`403`	4xx	Authenticated, not allowed
`404`	4xx	Unknown path or ID
`409`	4xx	State conflict (duplicate, version)
`422`	4xx	Validation failed (FastAPI / Pydantic)
`429`	4xx	Rate limited
`500`	5xx	Unexpected server error
`503`	5xx	Overload or maintenance

Codes are part of the API contract. **Caching & redirects** (`3xx` , e.g. `304`) — next slide.

Caching & redirects

Not a main topic in this course, but you will see these in production traces, CDNs, and auth flows — useful vocabulary to recognize.

3xx redirects

- The `Location` header tells the client where to continue; browsers follow it; HTTP libraries usually expose it for you to act on.
- `301` / `308` — permanent move (new canonical URL). `302` / `307` — temporary (e.g. OAuth login hand-off).

Caching & `304`

- `Cache-Control` and `ETag` control how long a response may be reused and how to re-check it cheaply.
- `304 Not Modified` — “your cached copy is still valid”; no new response body (common on repeated `GET`s behind CDNs).

Typical for static assets, public read APIs, and edge networks — less central than `2xx` / `4xx` for the small JSON APIs we focus on here.

Checkpoint: REST and HTTP

Q3

You want to create a new task in a REST-style API. Which option is the best fit?

A) `POST /tasks`

B) `GET /createTask`

C) `GET /tasks/new`

D) `DELETE /tasks` for dramatic effect

Show Answer

Q4

A client sends JSON with the wrong shape and FastAPI rejects it during validation. Which status code is most likely?

A) 200 OK

B) 404 Not Found

C) 422 Unprocessable Entity

D) 418 I'm a teapot, unless the JSON was brewed properly

Show Answer

Layered Architecture with FastAPI

Keep business logic reusable, transport shapes explicit, and web concerns at the edge

Hello, World! with FastAPI

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World"}
```

A plain dictionary becomes JSON automatically.

What to notice

- `app = FastAPI()` creates the ASGI application object
- `@app.get("/")` binds a path and HTTP method
- The function body contains normal Python code

Modern note: use `def` for simple sync code and `async def` when you are actually awaiting async I/O.

Run It, Explore It, Reuse the Schema

Modern setup commands

```
uv add fastapi "uvicorn[standard]"
uv run uvicorn main:app --reload
```

Useful local URLs

- `http://127.0.0.1:8000/` - your endpoint
- `http://127.0.0.1:8000/docs` - Swagger UI
- `http://127.0.0.1:8000/openapi.json` - raw schema

Why `/docs` is special

- FastAPI inspects routes and types
- It builds an OpenAPI schema automatically
- The docs page becomes a live, testable menu of your API

Fun FastAPI fact

The same OpenAPI schema behind `/docs` can also drive ReDoc pages and generated client SDKs.

Path parameters

Pieces of the URL become function arguments. Types in the signature drive parsing and validation.

```python
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

`/items/42` → FastAPI passes `42` as an `int`. Non-numeric paths yield `422`.

Try the same request two ways

`curl`

```
curl http://127.0.0.1:8000/items/42
```

Swagger UI

- Open `/docs` → find `GET /items/{item_id}`
- **Try it out** → enter `item_id` → **Execute**

Query parameters

After `?` in the URL: `name=value` pairs, separated by `&`. Optional parameters get defaults from the function signature.

```python
@app.get('/items')
def list_items(skip: int = 0, limit: int = 10):
    return {'skip': skip, 'limit': limit}
```

Omitted query args use the defaults (`0` and `10`).

Examples

```
curl "http://127.0.0.1:8000/items"
curl "http://127.0.0.1:8000/items?skip=5&limit=2"
```

In `/docs`, query fields appear as form inputs next to **Execute**.

Request body (JSON)

For `POST`, `PUT`, `PATCH`, … declare a Pydantic model. FastAPI reads JSON, validates it, and passes a model instance to your function.

```python
from pydantic import BaseModel

class ItemCreate(BaseModel):
    name: str
    price: float

@app.post("/items/")
def create_item(item: ItemCreate):
    return {"saved": item.model_dump()}
```

`curl` with JSON body

```bash
curl -X POST "http://127.0.0.1:8000/items/" \
  -H "Content-Type: application/json" \
  -d '{"name":"Milk","price":1.2}'
```

`/docs` shows a JSON editor for the body and builds the correct `Content-Type` header for you.

Path, query, and body together

One handler can take a path segment, query flags, and a JSON body. FastAPI decides each argument’s source from types and defaults.

```python
@app.put("/items/{item_id}")
def update_item(
    item_id: int,
    item: ItemCreate,
    dry_run: bool = False,
):
    if dry_run:
        return {"would_update": item_id, "data": item.model_dump()}
    return {"item_id": item_id, "saved": item.model_dump()}
```

`item_id` from path, `item` from body, `dry_run` from query.

curl

```
curl -X PUT \
  "http://127.0.0.1:8000/items/7?dry_run=true" \
  -H "Content-Type: application/json" \
  -d '{"name":"Tea","price":2.5}'
```

Swagger UI lists path, query, and body sections in one panel — useful to see the full contract.

When input is invalid

Bad types or missing required fields produce `422 Unprocessable Entity` with a structured `detail` list — before your route body runs.

```
# path: not an int
curl http://127.0.0.1:8000/items/not-a-number

# body: wrong types / missing field
curl -X POST "http://127.0.0.1:8000/items/" \
  -H "Content-Type: application/json" \
  -d '{"name":"Milk","price":"cheap"}'
```

Why this helps at intro level

- `/docs` shows the error response shape for failed validation
- Same rules apply whether the client is curl, a browser app, or tests
- You fix the contract (types and models), not ad-hoc string checks in every route

> Next: make parameters **explicitly typed** and attach **OpenAPI-friendly metadata** with `Annotated` + `Query` / `Path`.

Best practice: keep parameters typed

Use real Python types on path, query, and body parameters — not generic strings you parse by hand.

Why it matters

- **Validation** — bad values become `422` with a clear `detail` payload
- **OpenAPI** — `/docs` shows the right types, formats, and constraints
- **Tooling** — editors and type checkers understand your handler signatures

```python
# Prefer
@app.get('/users/{user_id}')
def get_user(user_id: int, active: bool = True):
    ...

# Avoid inventing ad-hoc parsing
def get_user(user_id: str):  # every caller sends str
    uid = int(user_id)  # manual, easy to get wrong
```

Let FastAPI coerce and validate; reserve manual parsing for rare edge cases.

`Annotated` + `Query( ... )`

Put extra rules and documentation on **query** parameters without hiding the real Python type. This is the style FastAPI recommends today.

```python
from typing import Annotated

from fastapi import Query

@app.get('/items')
def list_items(
    skip: Annotated[
        int,
        Query(ge=0, description='Offset into the list'),
    ] = 0,
    limit: Annotated[
        int,
        Query(le=100, ge=1, description='Page size'),
    ] = 10,
):
    return {'skip': skip, 'limit': limit}
```

What you gain

- `ge` / `le` — numeric bounds checked before your code runs
- `description` — appears in Swagger UI next to the field
- The parameter still reads as `int` for types and IDEs

Older tutorials use `limit: int = Query(10, le=100)`; `Annotated` keeps the type in front and scales better as metadata grows.

`Annotated` + `Path( ... )`

Path segments use the same pattern: type first, then `Path( ... )` for constraints and docs.

```python
from typing import Annotated

from fastapi import Path

@app.get('/items/{item_id}')
def read_item(
    item_id: Annotated[
        int,
        Path(ge=1, description='Primary key of the item'),
    ],
):
    return {'item_id': item_id}
```

Same idea as `Query`

- `Path` metadata shows up under the path parameter in `/docs`
- Combine with `Query` on the same route when you need both
- For bodies, Pydantic models stay the default; use `Body()` inside `Annotated` only when you need extra schema tweaks

Who Does What: FastAPI, Uvicorn, Pydantic, OpenAPI

FastAPI

Defines routes, parses parameters, calls dependencies, and builds responses.

Pydantic

Validates request data and helps serialize well-defined response shapes.

Uvicorn

The ASGI server that actually listens on a port and forwards HTTP traffic to the app.

OpenAPI

The machine-readable schema behind `/docs`, `/redoc`, and client generation.

Domain Model vs DTO

Domain model

- Represents business state and behavior
- Can have methods like `mark_complete()`
- Should not care about HTTP or JSON

DTO

- Defines the data shape coming in or going out
- Great for validation and serialization
- Usually has fields, not business behavior

> **Kitchen analogy:** the domain model is the meal being cooked; DTOs are the order slip and plated dish used for transport.

Our 4-Layer To-Do App

Layer 1

Web

`main.py`

HTTP in, JSON out

Layer 2

Service

`task_manager.py`

Coordinates use cases

Layer 3

Domain

`task.py`

Core business behavior

Layer 4

DTOs

`models.py`

Validated transport shapes

> The payoff is reuse: the same service/domain logic can power a CLI, a web API, and tests without being rewritten for each interface.

Modern DTOs with Pydantic v2

```python
from pydantic import BaseModel, ConfigDict

class TaskCreate(BaseModel):
    description: str

class TaskRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    description: str
    completed: bool
```

`from_attributes=True` is useful when you want to validate directly from an attribute-based domain object.

Why this helps

- Incoming JSON gets checked automatically
- Response data stays predictable
- Bad payloads fail early, before business logic runs

Fun Python tooling fact

Pydantic v2 is powered by Rust-based `pydantic-core`, one reason validation feels so fast.

Checkpoint: Architecture and FastAPI

Q5

Which layer should know about HTTP status codes such as `404` or `201`?

A) The web layer

B) The domain model

C) The database only

D) The layer with the fanciest filename

Show Answer

Q6

What does `uv run uvicorn main:app --reload` mean?

A) Run the object named `app` from `main.py` with auto-reload.

B) Compile Python into C and reboot the laptop.

C) Export the API directly to Kubernetes.

D) Ask Uvicorn politely to find the code on its own.

Show Answer

Dependency Injection and Testing

Make route handlers smaller, replace dependencies in tests, and keep the seams explicit

Dependency Injection: Tight vs Loose Coupling

Without DI

```python
@app.get("/tasks")
def list_tasks():
    manager = TaskManager()
    return manager.get_all_tasks()
```

With DI

```python
def get_manager():
    return task_manager

@app.get("/tasks")
def list_tasks(manager = Depends(get_manager)):
    return manager.get_all_tasks()
```

Why better? The route no longer decides how the manager is constructed, which makes substitution in tests much easier.

Modern Dependency Signatures with `Annotated`

```python
from typing import Annotated
from fastapi import Depends

ManagerDep = Annotated[TaskManager, Depends(get_manager)]

@app.get("/tasks", response_model=list[TaskRead])
def list_tasks(manager: ManagerDep):
    return manager.get_all_tasks()
```

The classic default-value style still works, but `Annotated` is the modern recommended form.

Why use it?

- Keeps the real type visible
- Makes reusable parameter aliases easy
- Works nicely with more complex parameter metadata later

Fun FastAPI fact

FastAPI normally caches dependency results per request, so the same dependency is usually resolved once and reused within that request.

The Decorator Is Part of the Contract

```python
@app.post(
    "/tasks",
    response_model=TaskRead,
    status_code=201,
)
def create_task(task_in: TaskCreate, manager: ManagerDep):
    return manager.create_task(task_in)
```

What the decorator communicates

- `POST /tasks`: create something in the collection
- `response_model=TaskRead`: output contract
- `status_code=201`: success means "created", not just "ok"

> The route body can return a domain object; FastAPI handles validation/serialization for the response contract.

Where Does Each Parameter Come From?

Path

`task_id: int`

Matches `{task_id}` in the URL and is converted to an integer.

Body

`task_in: TaskCreate`

FastAPI reads JSON from the request body and validates it with Pydantic.

Dependency

`manager: ManagerDep`

FastAPI calls the provider and injects the returned object.

Big idea: type hints and metadata are not just documentation here; FastAPI actively uses them to build the request-handling pipeline.

Tests need a clean seam: why DI matters

Production

One long-lived `TaskManager` holds tasks in memory for every request. Routes get it through `Depends(get_manager)`.

That is correct for a demo server — but tests must not all pile into the same store or mutate the object your real app would use.

What we want in tests

- A **fresh** manager per test (predictable empty state)
- No accidental coupling: test A's tasks must not leak into test B
- Room to swap a **fake** or **stub** later without rewriting routes

> **Dependency injection** gives FastAPI a named hook (`get_manager`). In tests we **replace** that hook — the route code stays unchanged.

Replace the provider: `dependency_overrides`

```python
from fastapi.testclient import TestClient
import pytest

@pytest.fixture
def client():
    test_manager = TaskManager()
    app.dependency_overrides[get_manager] = lambda: test_manager
    with TestClient(app) as test_client:
        yield test_client
    app.dependency_overrides.clear()
```

While the fixture is active, every route that depended on `get_manager` receives `test_manager`. After the test, overrides are cleared so the real app is not left in a weird state.

What `TestClient` does

- Calls your ASGI app in-process — no open port required
- Real request/response cycle: paths, query, body, status codes
- Works with the same `Depends` graph as production

The fixture's teardown (`clear()`) matters when you run many tests: each one gets a clean override window.

2026 Testing Notes: Async Tests and Lifespan

```python
from httpx import ASGITransport, AsyncClient

@pytest.mark.anyio
async def test_root():
    async with AsyncClient(
        transport=ASGITransport(app=app),
        base_url="http://test",
    ) as client:
        response = await client.get("/")
```

Use this pattern when the test itself is async and needs to await other async work too.

Modern rules of thumb

- Use `TestClient` for normal sync tests
- Use `AsyncClient` + `ASGITransport` for async tests
- Prefer app `lifespan` handlers over older `@app.on_event( ... )` patterns

Wrapping `TestClient(app)` in a `with` block also ensures startup and shutdown logic runs around the test session.

Checkpoint: Dependency Injection and Testing

Q7

Why is dependency injection especially helpful in tests?

A) It lets us replace real dependencies with controlled test doubles.

B) It makes HTTP disappear entirely.

C) It guarantees zero bugs forever.

D) It summons extra RAM from the cloud.

Show Answer

Q8

You are writing an `async def` test that also awaits other async database calls. Which client pattern fits best?

A) `AsyncClient` with `ASGITransport(app=app)`

B) A random browser tab and optimism

C) `print()` statements only

D) `DELETE /tests`

Show Answer

Lecture Summary

Web mindset

- Services stay alive and listen
- Clients and servers talk via HTTP
- Each request should stand on its own

API contract

- Paths name resources
- Methods express actions
- Status codes and DTOs make expectations explicit

Maintainability

- Keep HTTP at the edge
- Inject dependencies instead of hard-wiring them
- Test through the API surface with controlled overrides

Core message: clean contracts, clear boundaries, and testable seams make web APIs easier to evolve safely.

Thank You for Your Attention!

"APIs should be easy to use and hard to misuse."

— Joshua Bloch, Effective Java